

OOPs

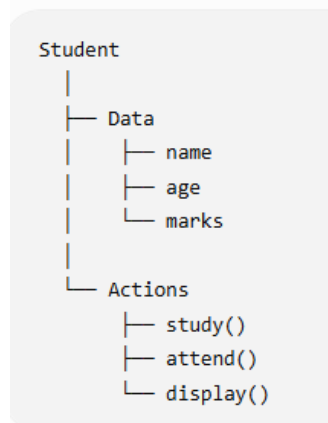
OOPs stands for **Object-Oriented Programming System** (commonly called **Object-Oriented Programming**).

Object-Oriented Programming (OOP) is a programming paradigm that organizes software design around objects, which represent real-world entities. These objects encapsulate **data (attributes)** and **behavior (methods)**, making the code more modular and reusable.

It is a way of writing Python programs using **classes and objects**.

OOP is a programming approach where we organize data and functions together inside classes and create objects from those classes.

Think about a **Student**:



Example:

class Student:

```
def __init__(self, name, age):
    self.name = name
    self.age = age
```

```
def display(self):
    print(self.name, self.age)
```

```
s1 = Student("Abhinav", 22)
s1.display()
```

Here:

- Student → **Class**
- s1 → **Object**
- name, age → **Data/Attributes**
- display() → **Method**

Why do we use OOP?

- **Organize** large programs
- **Reuse** code
- **Protect** data
- **Maintain** code easily
- Build complex applications more easily

Key Concepts of OOP in Python

1. **Class** – A blueprint for creating objects.
2. **Object** – An instance of a class.
3. **Encapsulation** – Hiding the internal details of an object and restricting direct access to some of its components.
4. **Abstraction** – Hiding complex implementation details and exposing only the necessary parts.
5. **Inheritance** – A mechanism that allows a new class to derive properties and behavior from an existing class.
6. **Polymorphism** – The ability to take multiple forms, e.g., methods with the same name but different implementations in different classes.

OOP follows four main principles:

1. **Encapsulation** – Hiding the internal state of an object and restricting access to it.
2. **Abstraction** – Hiding complex implementation details and exposing only the necessary parts.
3. **Inheritance** – Enabling new classes to derive properties and methods from existing ones.
4. **Polymorphism** – Allowing the same interface to be used for different underlying data types.

Difference Between OOP in Python and Other Languages

Feature	Python	Java	C++	C#
Syntax	Simple & Dynamic	Strict & Verbose	Complex	Similar to Java
Multiple Inheritance	✓ Yes	✗ No (only interfaces)	✓ Yes	✗ No (only interfaces)
Encapsulation	✓ Uses <code>_</code> and <code>__</code> (weak enforcement)	✓ Uses <code>private</code> , <code>protected</code> , <code>public</code>	✓ Strong encapsulation	✓ Strong encapsulation
Polymorphism	✓ Dynamic typing	✓ Strict typing	✓ Compile-time & runtime	✓ Strong polymorphism
Memory Management	✓ Automatic (Garbage Collection)	✓ Automatic (JVM)	✗ Manual	✓ Automatic
Performance	Slower (interpreted)	Faster (compiled) ↓	Very fast	Faster than Python

Benefits of OOP

- ✓ **1. Code Reusability (DRY Principle)**
 - Inheritance allows reusing existing code without rewriting it.
 - Example: A Car class can be reused in multiple projects.
- ✓ **2. Modularity (Easier Code Organization)**
 - Classes help break down large code into smaller, manageable parts.
 - Example: A BankAccount class can have different methods like `deposit()` and `withdraw()`.
- ✓ **3. Encapsulation (Data Security)**
 - Prevents direct modification of sensitive data.
 - Example: A private `__balance` attribute in a BankAccount class.
- ✓ **4. Scalability & Flexibility**
 - Polymorphism allows the same method to work for different types.
 - Example: A `make_sound()` method can work for both Dog and Cat classes.
- ✓ **5. Real-World Representation**
 - Objects model real-world entities, making software design intuitive.

- Example: Car, Employee, Customer, etc.
-

Class

A **class** is a **blueprint or template** used to create objects in Python.

A **class** in Python is a blueprint or template for creating objects. It defines attributes (variables) and methods (functions) that the objects will have.

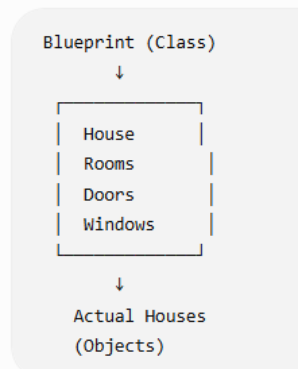
Syntax of a Class in Python

```
class ClassName:
    # Constructor (optional)
    def __init__(self, param1, param2):
        self.param1 = param1
        self.param2 = param2

    # Method
    def method_name(self):
        print("Method executed")
```

- class keyword is used to define a class.
- `__init__()` is a special method (constructor) that initializes object attributes.
- self represents the instance of the class.

Think about a **house blueprint** 🏠:



The **blueprint is not the actual house**. It describes how the house should be built.

Similarly:

Class = Blueprint

Object = Actual thing created from the blueprint

Example

```
class Student:
    name = "Abhinav"
    age = 22
```

Here, Student is a **class**.

We can create an object:

```
s1 = Student()
```

```
print(s1.name)
```

```
print(s1.age)
```

Output:
Abhinav
22

Class with a method

A class can contain both **data and functions**:

```
class Student:
    name = "Abhinav"

    def study(self):
        print("Student is studying")
```

```
s1 = Student()
print(s1.name)
s1.study()
```

Output:
Abhinav
Student is studying

Here study() is called a **method** because it is a function inside a class

Object

An object is simply a collection of data(variable) and Methods(functions) that act on those data. Similarly, a class is a blueprint of an object.

An object (instance) is an instantiation of a class. When class is defined, only the description for the object is defined. Therefore, no memory or storage is allocated.

There is no memory allocation until we create its object. The Object or Instance Contains real Data or Information

Example:

```
class Abhi():
    def Output(self):
        print("Hello World")
```

```
vivo=Abhi()
vivo.Output()
```

Multiple objects

One class can create many objects:

```
class Student:
    pass
```

```
s1 = Student()
s2 = Student()
```

```
s3 = Student()
```

Real-world example

Class → Car

Objects → BMW, Audi, Toyota

The **Car class** describes what a car can have/do, while individual cars are **objects**.

Example-2:

```
class Kiran(): # class keyword class name
    a=10 # data member
    def Output(self): # method(self)
        print(self.a)
obj=Kiran() # declaring of object
obj.Output() # using object we can call the methods
```

self Keyword:

We can access the attribute and methods of the class (current class only)

Example:

```
class Abhi():
    def Output(self):
        print("Hello World")
```

```
vivo=Abhi()
vivo.Output()
```

Example-2:

```
class Abhi():
    a=10
    def show(self):
        print("this is class")
# obj name=class name()
sai=Abhi()
print(sai.a)
sai.show()
```

Example-3:

```
class Abhi():
    a=10 #attribute
    def display(self):
        print(self.a)
```

```
A1=Abhi()
A2=Abhi()
A1.display()
A2.display()
```

constructor

- Constructor is a special method that is Automatically called when you create an object.
- Constructors are generally used for instantiating an object.
- The task of constructors is to initialize(assign values)

- Constructor = Object's initial setup
- In Python the `__init__()` method is called the constructor and is always called when an object is created
- doesn't support multiple constructor

Example-1:

For example,
when you create a student, you may want to give the student a **name and age immediately**.

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
s1 = Student("Abhinav", 22)
print(s1.name)
print(s1.age)
```

Output:
Abhinav
22

Why do we use a constructor?

We use it to **initialize an object's data when the object is created**.

For example:

```
class Employee:

    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
```

```
e1 = Employee("Abhinav", 30000)
e2 = Employee("Rahul", 40000)
```

Now:

```
e1 → name = Abhinav, salary = 30000
e2 → name = Rahul, salary = 40000
```

Important terms

```
def __init__(self, name, age):
```

- `__init__` → constructor/initializer method
- `self` → current object
- `name, age` → parameters
- `self.name, self.age` → object attributes

Example-2:

```
class Abhi():
    def __init__(self,a,b,c):
```

```

        self.A1=a
        self.B1=b
        self.C1=c

    def Output(self):
        print(self.A1, self.B1, self.C1)
p=Abhi(1,2,3)
p.Output()

```

Example-3:

```

class Mobiles():
    def __init__(self,Mobile_name,Mobile_Ram,Mobile_battery,Mobile_Price):
        self.a=Mobile_name
        self.c=Mobile_Ram
        self.d=Mobile_battery
        self.e=Mobile_Price

    def Mobile_Data(self):
        print("Mobile Name:",self.a)
        print("Mobile Ram:",self.c)
        print("Mobile Battery:",self.d)
        print("Mobile Price:",self.e)

name=input("Enter the Mobile Name:")
ram=input("Enter the Mobile Ram:")
bat=input("Enter the Mobile Battery:")
Price=float(input("Enter the Mobile Price:"))

Mobile_obj=Mobiles(name,ram,bat,Price)
Mobile_obj.Mobile_Data()

```

Example-4:

```

class Bikes():
    def __init__(self, bike_name, bike_cc, millage):
        self.a=bike_name
        self.b=bike_cc
        self.c=millage

    def Bike_Data(self):
        print("Bike Name:", self.a)
        print("Bike CC:", self.b)
        print("Bike Millage:", self.c)

bike_name = input("Enter Bike-1 Name")
bike_cc = input("Enter Bike-1 CC")
millage = input("Enter Bike-1 Millage")
Bike1 = Bikes(bike_name, bike_cc, millage)
Bike1.Bike_Data()

```

```
bike_name = input("Enter Bike-2 Name")
bike_cc = input("Enter Bike-2 CC")
millage = input("Enter Bike-2 Millage")

Bike2 = Bikes(bike_name, bike_cc, millage)
Bike2.Bike_Data()
```

Inheritance

Inheritance is a fundamental concept of Object-Oriented Programming (OOP) that allows one class (**child/subclass**) to inherit the attributes and methods of another class (**parent/base class**). It enables **code reusability**, **hierarchical class organization**, and **extensibility** in programs.

Inheritance is an OOP concept where a **child class gets the properties and methods of a parent class**.

Types of Inheritance in Python

Python supports five types of inheritance:

1. **Single Inheritance** – One child class inherits from one parent class.
 2. **Multiple Inheritance** – A child class inherits from multiple parent classes.
 3. **Multilevel Inheritance** – A child class inherits from another child class (grandparent → parent → child).
 4. **Hierarchical Inheritance** – Multiple child classes inherit from a single parent class.
 5. **Hybrid Inheritance** – A combination of two or more types of inheritance.
-

Example:

```
class Parent:
    def eat(self):
        print("Eating")

class Child(Parent):
    def play(self):
        print("Playing")
```

Now create an object of Child:

```
c = Child()

c.eat()
c.play()
```

Output:

```
Eating
Playing
```

Notice something important:

eat() was defined inside Parent, but the Child object can use it. That's **inheritance**.

Why do we use inheritance

1. Code Reusability
 2. Avoid duplicate code
-

Types of Inheritance

Python supports several forms.

1. Single Inheritance

One parent → One child

Example:

```
class Parent:
    def output(self):
        print('this is parent class')
class Child(Parent):
    def outputChild(self): # output
        print('this is child class')
i=Child()
i.output()
i.outputChild()
```

2. Multilevel Inheritance

Grandparent → Parent → Child

Example:

```
class Father:
    def output(self):
        print('this is parent class')
class Mother:
    def outputM(self):
        print('this is mother class')
class Child(Father,Mother):
    def outputChild(self):
        print('this is child class')
ice=Child()
ice.output()
ice.outputM()
ice.outputChild()
```

3. Multiple Inheritance

One child → Multiple parents

Example:

```
class Father:
    def output(self):
        print('this is parent class')
class Mother:
    def outputM(self):
        print('this is mother class')
class Child(Father,Mother):
    def outputChild(self):
        print('this is child class')
ice=Child()
ice.output()
```

```
ice.outputM()  
ice.outputChild()
```

4. Hierarchical Inheritance

One parent → Multiple children

Example:

```
class Father:#100cr  
    def output(self):  
        print('this is father class')  
  
class Child1(Father):#50cr  
    def outputf(self):  
        print('this is child 1 class')  
  
class Child2(Father):#50cr  
    def outputChild(self):  
        print('this is child 2 class')  
ice=Child1()  
cream=Child2()  
ice.output() #child 1 of parent  
ice.outputf()  
cream.output() # child 2 of parent  
cream.outputChild() # child 2
```

Polymorphism

Polymorphism is an OOP concept where the **same method or operation can behave differently depending on the object.**

Easy meaning of the word

Poly = Many

Morph = Forms

So:

Polymorphism = One thing having many forms.

Example

Both Dog and Cat have a method called sound(), but they behave differently.

```
class Dog:  
    def sound(self):  
        print("Bark")  
  
class Cat:  
    def sound(self):  
        print("Meow")  
  
dog = Dog()  
cat = Cat()
```

```
dog.sound()  
cat.sound()
```

Output

Bark

Meow

Here, the method name is the same:

sound()

But the behavior is different:

Dog → Bark

Cat → Meow

This is **polymorphism**.

Why do we use polymorphism?

Polymorphism helps us write:

1. Flexible code

```
def make_sound(animal):  
    animal.sound()
```

This function can work with different objects:

```
make_sound(Dog())
```

```
make_sound(Cat())
```

2. Reusable code

Instead of creating separate functions for every object, one common operation can work with multiple object types.

Method Overloading

Method overloading means having **multiple methods with the same name but different parameters**.

For example, in some programming languages:

```
add(a, b)
```

```
add(a, b, c)
```

Both methods are named add, but they accept different numbers of parameters.

Important: Python does not support traditional method overloading

If you define the same method name multiple times in a Python class, the **last definition replaces the previous one**.

class Calculator:

```
    def add(self, a, b):  
        print(a + b)
```

```
    def add(self, a, b, c):  
        print(a + b + c)
```

Now:

```
c = Calculator()
```

```
c.add(10, 20, 30)
```

Python uses only the last add() method.

The first one is overwritten.

How do we achieve overloading-like behavior in Python?

1. Using Default Arguments

class Calculator:

```
def add(self, a, b, c=0):  
    print(a + b + c)
```

Now:

```
c = Calculator()
```

```
c.add(10, 20)
```

```
c.add(10, 20, 30)
```

Output:

30

60

2. Using *args

class Calculator:

```
def add(self, *numbers):  
    print(sum(numbers))
```

Now:

```
c = Calculator()
```

```
c.add(10, 20)
```

```
c.add(10, 20, 30)
```

```
c.add(10, 20, 30, 40)
```

Output:

30

60

100

Method Overloading vs Method Overriding

Method Overloading	Method Overriding
Same class	Parent and child classes
Same method name	Same method name
Different parameters	Same method signature is commonly used
Python doesn't support traditional form	Python supports it
Achieved using *args or defaults	Child changes parent method behavior

Method Overriding

Method overriding happens when a **child class provides its own implementation of a method that already exists in the parent class.**

Parent has a method → Child changes that method

Example

class Animal:

```
def sound(self):  
    print("Animal makes a sound")
```

```
class Dog(Animal):
    def sound(self):
        print("Dog barks")
```

Now:

```
dog = Dog()
dog.sound()
```

Output

Dog barks

What happened here?

The parent class:

```
class Animal:
    def sound(self):
        print("Animal makes a sound")
```

has a method called:

sound()

The child class Dog also creates a method with the **same name**:

```
class Dog(Animal):
    def sound(self):
        print("Dog barks")
```

So the child's method **overrides** the parent's method.

super()

super() is used to **access the parent class** from a child class.

Example 1: Calling Parent Method

```
class Parent:
    def show(self):
        print("Parent method")

class Child(Parent):
    def show(self):
        super().show() # Calls parent method
        print("Child method")
```

```
c = Child()
c.show()
```

Output

Parent method
Child method

Example 2: Using super() with Constructor

This is a very common use of super().

```
class Parent:
    def __init__(self, name):
        self.name = name

class Child(Parent):
    def __init__(self, name, age):
        super().__init__(name)
        self.age = age

c = Child("Abhinav", 22)

print(c.name)
print(c.age)
```

Output

Abhinav
22

Example:

```
class Methodoverri:
    def display(self):
        print("this is parent class")

class Child(Methodoverri):
    def display(self):
        print("this is child class")
        super().display()

obj=Child()
obj.display()
```

Encapsulation

Encapsulation is an OOP concept where we **combine data (variables) and methods (functions) inside a class and control access to the data.**

Encapsulation = Data + Methods together + Control data access

Real-life example 🏠

Think about a **bank account**.

Your balance should not be freely changed from anywhere in the program.

Instead:

Bank Account

```
|
|— Balance 🔒
|
|— deposit()
|— withdraw()
|— get_balance()
```

You use methods to interact with the balance.

Example in Python

```
class BankAccount:
```

```
def __init__(self, balance):
    self.__balance = balance

def deposit(self, amount):
    self.__balance += amount

def get_balance(self):
    return self.__balance

account = BankAccount(1000)
account.deposit(500)

print(account.get_balance())
```

Output

1500

What does `__balance` mean?

`self.__balance`

The double underscore (`__`) indicates a **private attribute convention with name mangling** in Python. It is used to prevent normal direct access such as:

`account.__balance`

Instead, access or modify the value through methods:

`account.deposit(500)`

`print(account.get_balance())`

Access levels in Python

Python commonly uses these naming conventions:

`public_var` # Public

`_protected_var` # Protected convention

`__private_var` # Private-like via name mangling

⚠ Important: Python does not enforce access modifiers exactly like languages such as Java. These are mainly conventions, and `__name` uses **name mangling**, not absolute privacy.

Example-1:

```
class GFather:
    def __init__(self,a):
        self._y=a
        print(self._y)
class Father(GFather):
    def display1(self):
        print(self._y)
class Child2(Father):
    def display2(self):
        print("child2",self._y)
obj=Child2(12)
obj.display2()
obj.display1()
```

Output:

```
12
child2 12
12
```

Example-2:

```
class GFather:
    def __init__(self,a):
        self.__y=a
        print(self.__y)
class Father(GFather):
    def display1(self):
        print(self.__y)
class Child2(Father):
    def display2(self):
        print("child2",self.__y)
obj=Child2(12)
obj.display2()
obj.display1()
```

Output:

```
12
Traceback (most recent call last):
  File "e:\Python Life\1. Python Full Stack\Day-11 Opps and ML intro\task-31 oops practice.py", line
180, in <module>
    obj.display2()
  File "e:\Python Life\1. Python Full Stack\Day-11 Opps and ML intro\task-31 oops practice.py", line
178, in display2
    print("child2",self.__y)
          ^^^^^^^^^
AttributeError: 'Child2' object has no attribute '_Child2__y'
```

Abstraction

Abstraction means **hiding unnecessary implementation details and showing only the important features to the user.**

Abstraction = Hide complexity and show only what is necessary.

Real-life Example: Car 🚗

When you drive a car:

You press Start



Car starts

You know **how to use the car**, but you don't need to know all the internal engine operations.

What you see

What is hidden

Start button

Engine process

Steering wheel

Internal mechanism

Accelerator

Fuel system

This is **abstraction**.

Abstraction in Python

Python provides abstraction using the abc module.

abc means:

Abstract Base Class

Example:

```
from abc import ABC, abstractmethod
```

```
class Animal(ABC):
```

```
    @abstractmethod
```

```
    def sound(self):
```

```
        pass
```

Here:

- Animal is an **abstract class**
- sound() is an **abstract method**

What is an Abstract Method?

An **abstract method** is a method that has a declaration but does not provide the complete implementation in the abstract class.

```
@abstractmethod
```

```
def sound(self):
```

```
    pass
```

It tells child classes:

"You must create your own sound() method."

Child Classes

```
from abc import ABC, abstractmethod
```

```
class Animal(ABC):
```

```
    @abstractmethod
```

```
    def sound(self):
```

```
        pass
```

```
class Dog(Animal):
```

```
    def sound(self):
```

```
        print("Bark")
```

```
class Cat(Animal):
```

```
    def sound(self):
```

```
        print("Meow")
```

Now:

```
dog = Dog()
```

```
cat = Cat()
```

```
dog.sound()
```

```
cat.sound()
```

Output

Bark

Meow

Example:

```
from abc import ABC, abstractmethod
```

```
class Car(ABC):
```

```
    @abstractmethod
```

```
    def mileage(self):
```

```
        pass
```

```
class Tesla(Car):
```

```
    def mileage(self):
```

```
        print("The mileage is 30kmph")
```

```
class Suzuki(Car):
```

```
    def mileage(self):
```

```
        print("The mileage is 25kmph ")
```

```
class Duster(Car):
```

```
    def mileage(self):
```

```
        print("The mileage is 24kmph ")
```

```
class Renault(Car):
```

```
    def mileage(self):
```

```
        print("The mileage is 27kmph ")
```

```
# Driver code
```

```
t= Tesla ()
```

```
t.mileage()
```

```
r = Renault()
```

```
r.mileage()
```

```
s = Suzuki()
```

```
s.mileage()
```

```
d = Duster()
```

```
d.mileage()
```